

Welcome to

Lab4: Neural Networks

Date: 04.03.2026

Who I am: Ceren Gok - PhD student working on Continual Learning and TA for MLE course 2026!

What is these Labs are about: Guided-coding exercises, Try to remember what we learned from the lectures, Sharing our opinions with our peers, aaannd competing with fun way !

The Structure of Slides: Historical Facts, Fun Facts, Revisiting Theories, Interview Questions, Live Coding Exercise for you.

Welcome to

Lab4: Neural Networks

Intended Learning Outcomes:

- Why NN matters
- Build MLP and use it for image recognition
- Understand the core component: activation
- Understand the core component: optimizer
- Understand the core component: learning rate
- Understand the core component: weight initialization

Welcome to

Lab4: Neural Networks

Lets start with a Game and see how much we remember from lecture notes!

The vanishing gradient problem is completely solved by using ReLU.



[link for the game](#)



Historical Fact: The Rise, Fall, and Return of Neural Networks



Neural networks were invented BEFORE many classical ML methods.

Historical Fact: The Rise, Fall, and Return of Neural Networks



Neural networks were invented BEFORE many classical ML methods.

- kNN 1951
- Logistic Regression 1950s
- Decision Trees 1960s–1980
- 1943 First artificial neuron
- 1958 Perceptron

Historical Fact: The Rise, Fall, and Return of Neural Networks



Neural networks were invented BEFORE many classical ML methods.

- kNN 1951
- Logistic Regression 1950s
- Decision Trees 1960s–1980
- 1943 First artificial neuron
- 1958 Perceptron

But... they were not practical for many years.

Historical Fact: The Rise, Fall, and Return of Neural Networks



Problem1: Single-layer perceptron cannot solve non linear problem

Historical Fact: The Rise, Fall, and Return of Neural Networks



Problem1: Single-layer perceptron cannot solve non linear problem

Problem2: Multilayer networks existed in theory, but no efficient training algorithm

Historical Fact: The Rise, Fall, and Return of Neural Networks



Problem1: Single-layer perceptron cannot solve non linear problem

Problem2: Multilayer networks existed in theory, but no efficient training algorithm

There were works going on with backpropagation to train NN efficiently

Historical Fact: The Rise, Fall, and Return of Neural Networks



Problem1: Single-layer perceptron cannot solve non linear problem

Problem2: Multilayer networks existed in theory, but no efficient training algorithm

There were works going on with backpropagation to train NN efficiently

In 1986, backpropagation became popular again
Scientist show that it can solve non linear problem successfully.

Historical Fact: The Rise, Fall, and Return of Neural Networks



Problem1: Single-layer perceptron cannot solve non linear problem

Problem2: Multilayer networks existed in theory, but no efficient training algorithm

There were works going on with backpropagation to train NN efficiently

In 1986, backpropagation became popular again
Scientist show that it can solve non linear problem successfully.

But still...
They were not dominant yet.

Historical Fact: The Rise, Fall, and Return of Neural Networks



Why not popular even after backpropagation?

1. Datasets were small (No ImageNet).
2. No GPUs.
3. Still problems with training, no ReLu, poor initialization

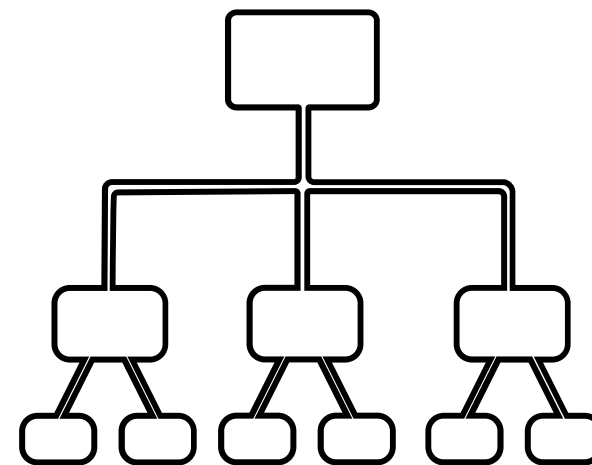
Historical Fact: The Rise, Fall, and Return of Neural Networks



Why not popular even after backpropagation?

1. Datasets were small (No ImageNet).
2. No GPUs.
3. Still problems with training, no ReLu, poor initialization

So it was the Era of Traditional Models



Historical Fact: The Rise, Fall, and Return of Neural Networks

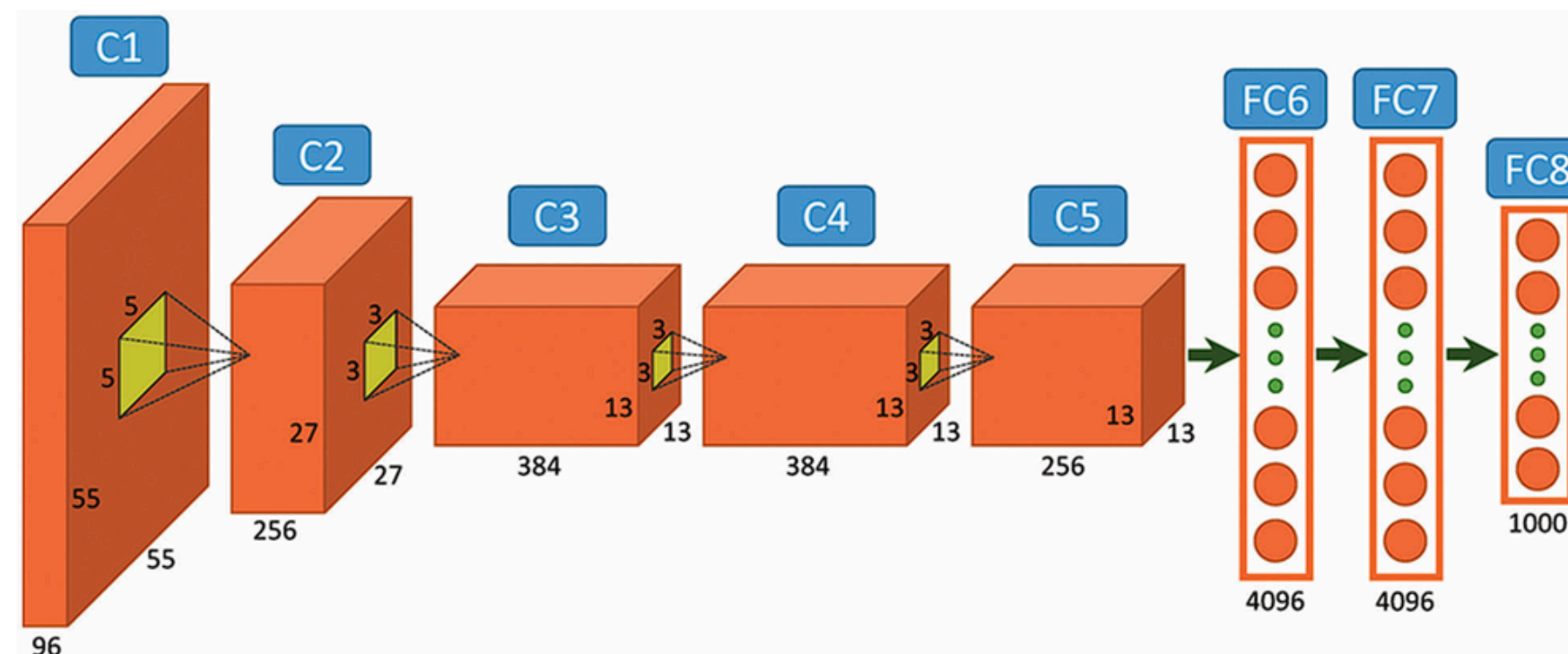


Big turning point was in 2012

CVPR Challenge with ImageNet Dataset

Alex Krizhevsky,
Ilya Sutskever,
Geoffrey Hinton

→ AlexNet model (CNN based architecture) → They beat the traditional model's performance



What Exactly Is the Advantage of NN?

Linear vs NonLinear and Hiearchical Learning

Traditional ML

Often learn via linear functions
(logistic regression)
or
via splits (decision tree)

NN

Learn nonlinear transformations
and

Build hierarchical representations

Layer1: Transform input space

Layer2: Reshape it

Layer3: Make it linearly seperable

What Exactly Is the Advantage of NN?

Linear vs NonLinear and Hieararchical Learning

Traditional ML

Often learn via linear functions
(logistic regression)
or
via splits (decision tree)

NN

Learn nonlinear transformations
and

Build hierarchical representations

Layer1: Transform input space

Layer2: Reshape it

Layer3: Make it linearly seperable

Representation Learning

Logistic regression learns decision boundary directly on **raw features**.

NN learns **internal representations**.
Learns a new feature space first.
Then classifies.

This is a huge conceptual difference.

What Exactly Is the Advantage of NN?

Linear vs NonLinear and Hiearchical Learning

Traditional ML

Often learn via linear functions
(logistic regression)
or
via splits (decision tree)

NN

Learn nonlinear transformations
and
Build hierarchical representations
Layer1: Transform input space
Layer2: Reshape it
Layer3: Make it linearly seperable

Representation Learning

Logistic regression learns decision boundary directly on **raw features**.

NN learns **internal representations**.
Learns a new feature space first.
Then classifies.

This is a huge conceptual difference.

Complexity of Tasks

$32 \times 32 \times 3 = 3072$ features for CIFAR
maybe 150,000+ for larger images

Decision trees:

Need enormous depth
Become unstable
Overfit easily

Our Data for Today: CIFAR10



Who: Canadian Research Institute
Geoffrey Hinton, Alex Krizhevsky, Vinod Nair,

Why: Benchmarking Deep Generative Models but it became useful for
image recognition

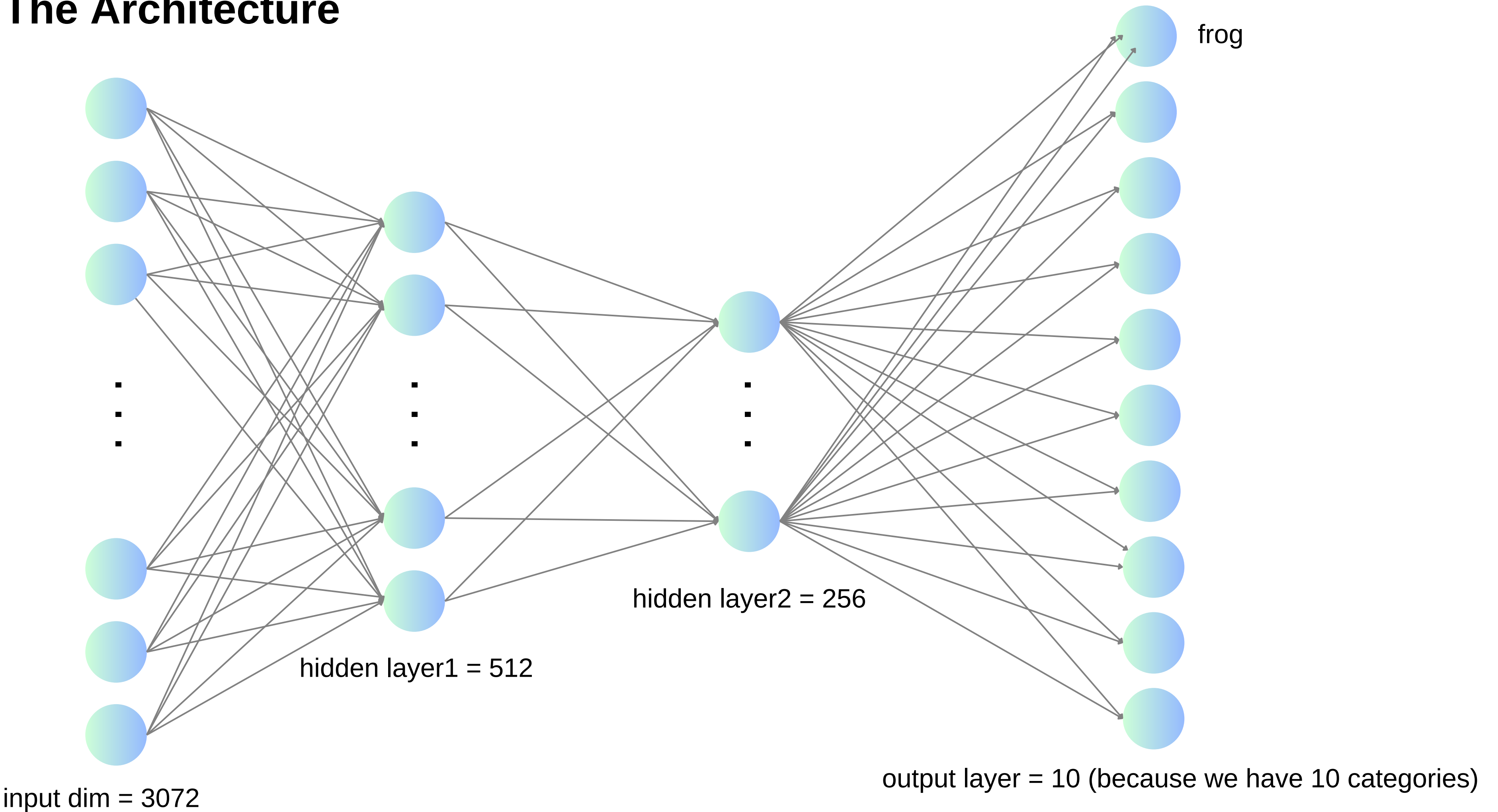
What: 50,000 training 10,000 test samples
10 categories
32 x 32 RGB images

Our Mission Today: Create the Best Recipe for NN Soup

We have core ingredients to decide:

- The Architecture

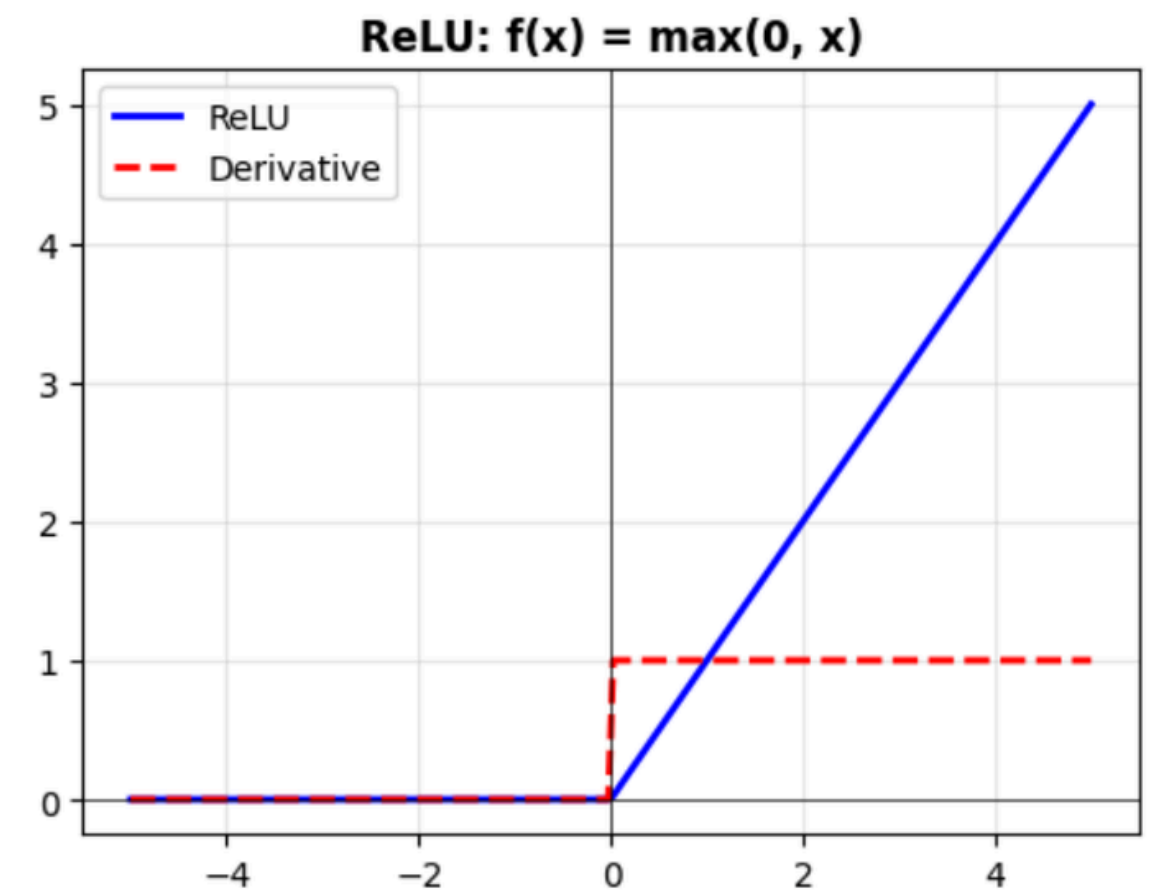
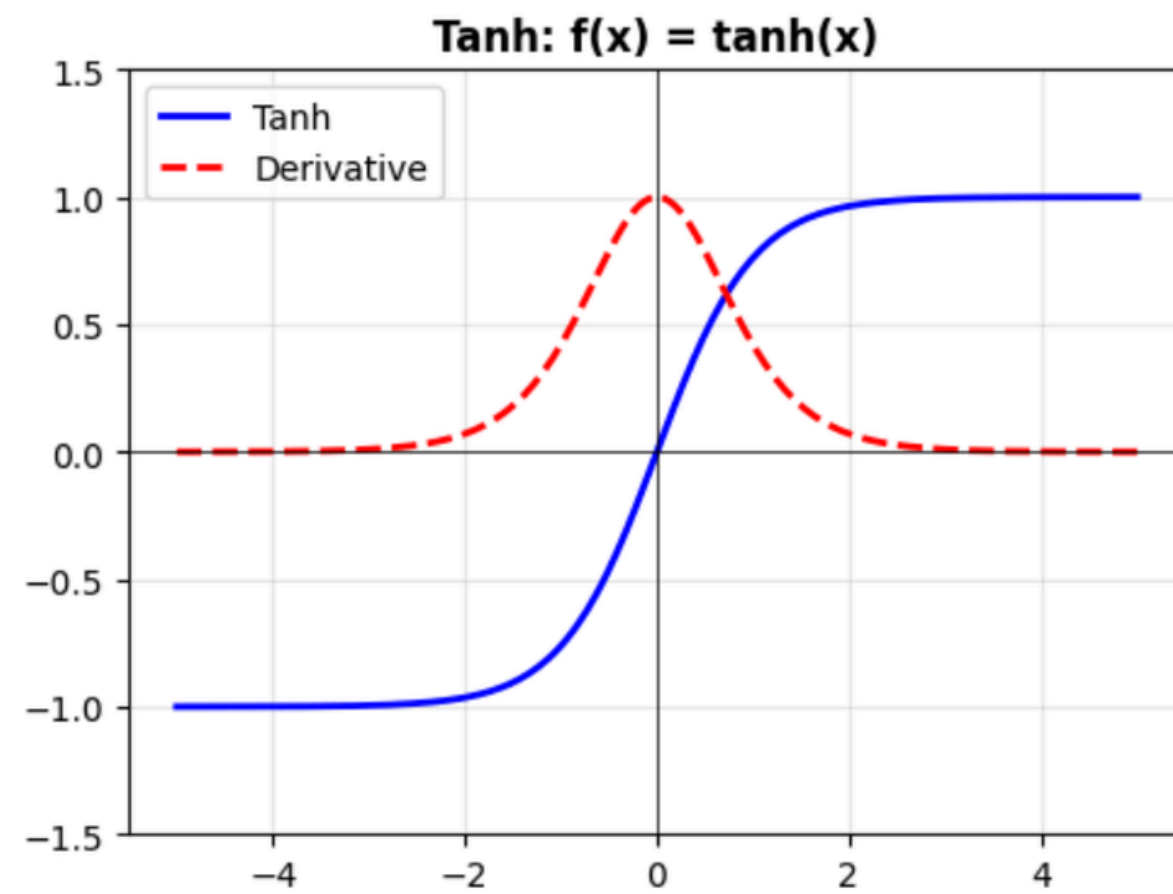
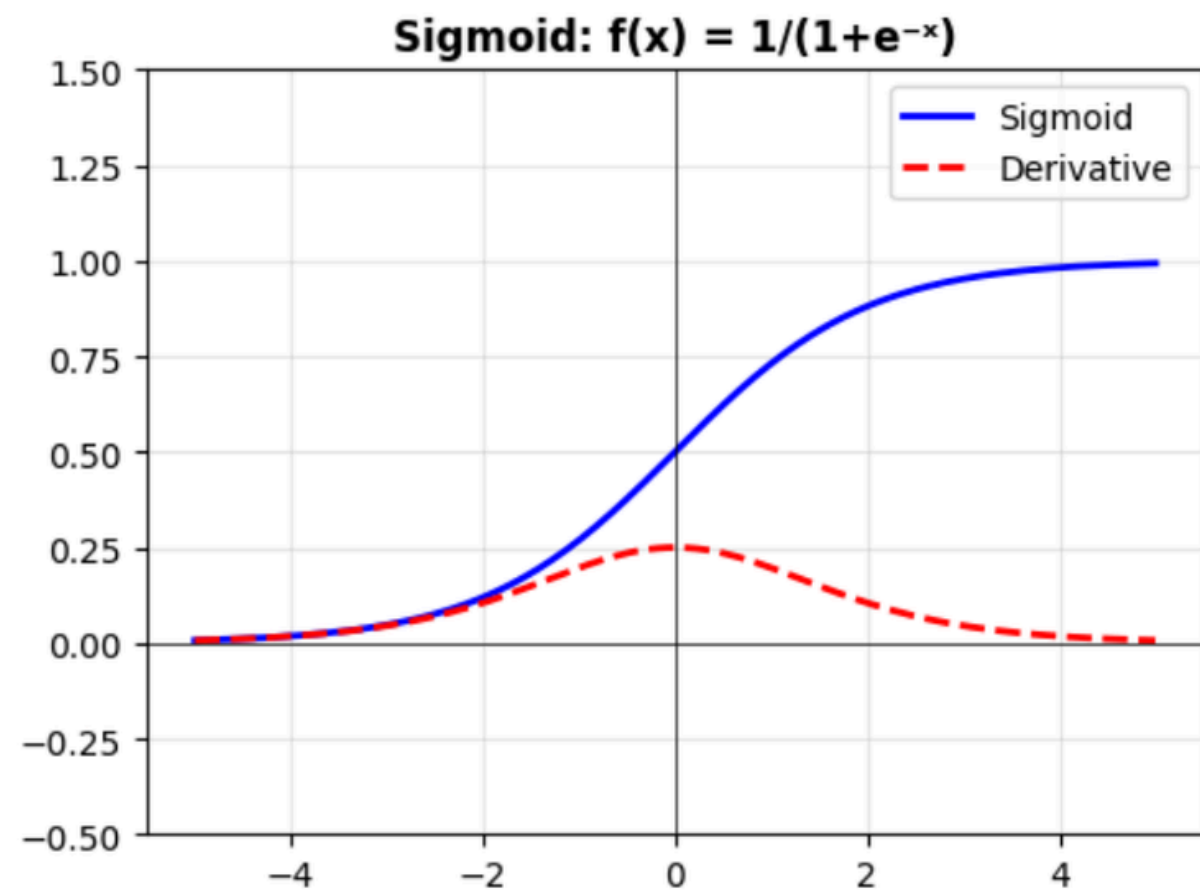
The Architecture



Our Mission Today: Create the Best Recipe for NN Soup

We have core ingredients to decide:

- The Architecture
- Activation Function



Interview Moment: The Silent Killer Problem with Activations



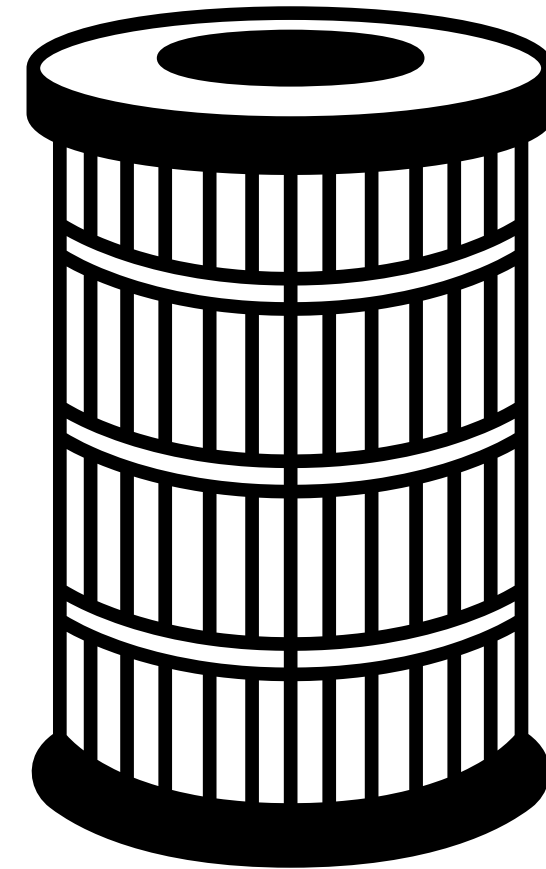
You joined the Dyson R&D department. The team is building a **image classification** system for air quality particle identification.

The current prototype is a neural network with 8 hidden layers.

In a design review, your teammate confidently says:
“Let's use **Sigmoid** activations across **all hidden layers**.
It's **smooth**, well-understood, and outputs nice probability-like values.”

You take a breath and push back:
“I think we should use **ReLU** instead.”

How can you explain **why ReLU** is generally **superior** to **Sigmoid** for **hidden** layers in **deep neural networks**?



Interview Moment: The Silent Killer Problem with Activations



ReLU is better because it's **faster**.

Sigmoid is **slow** and old-fashioned.

Everyone uses **ReLU** now because it's the modern choice.

Also, **ReLU** is **simpler** to compute.

Weak Answer

Interview Moment: The Silent Killer Problem with Activations



ReLU is better because it's **faster**.

Sigmoid is **slow** and old-fashioned.

Everyone uses **ReLU** now because it's the modern choice.

Also, **ReLU** is **simpler** to compute.

Weak Answer

The core issue is the **vanishing gradient** problem in deep networks.

With sigmoid activation function we risk the **gradients** to become **very very small**. The **early layers** receive almost **zero gradient**. And this is what we call vanishing gradient.

If gradients are nearly zero → weights barely update.

If weights barely update → those layers don't learn.

If early layers don't learn → the whole network struggles.

ReLU behaves **differently**. For positive inputs, its gradient is 1.

That means when **gradients** flow backward, they **stay strong** instead of shrinking.

Gradients remain **large** enough.

Early layers receive meaningful updates.

ReLU can **suffer** from the “**dying ReLU**” problem:

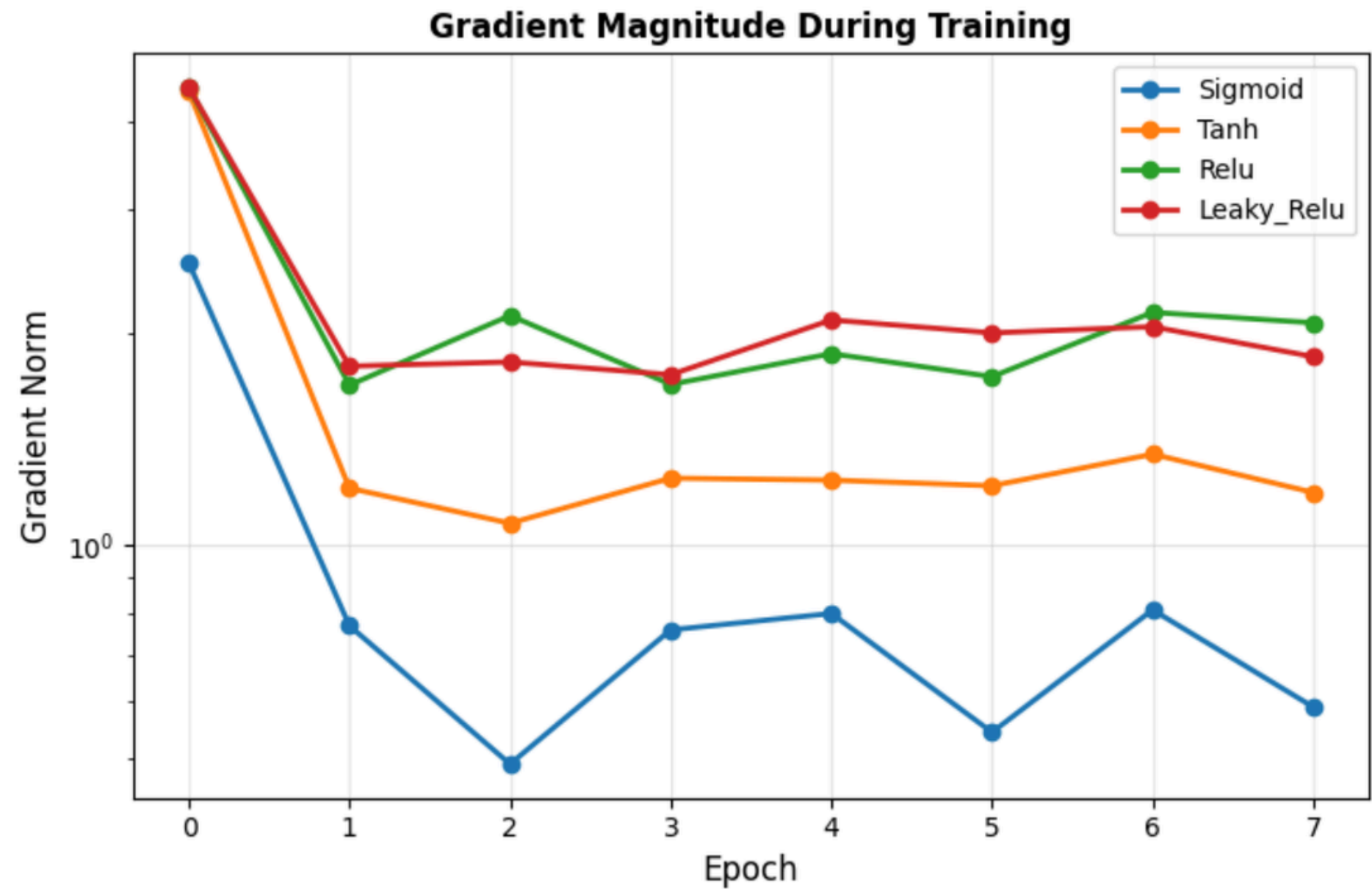
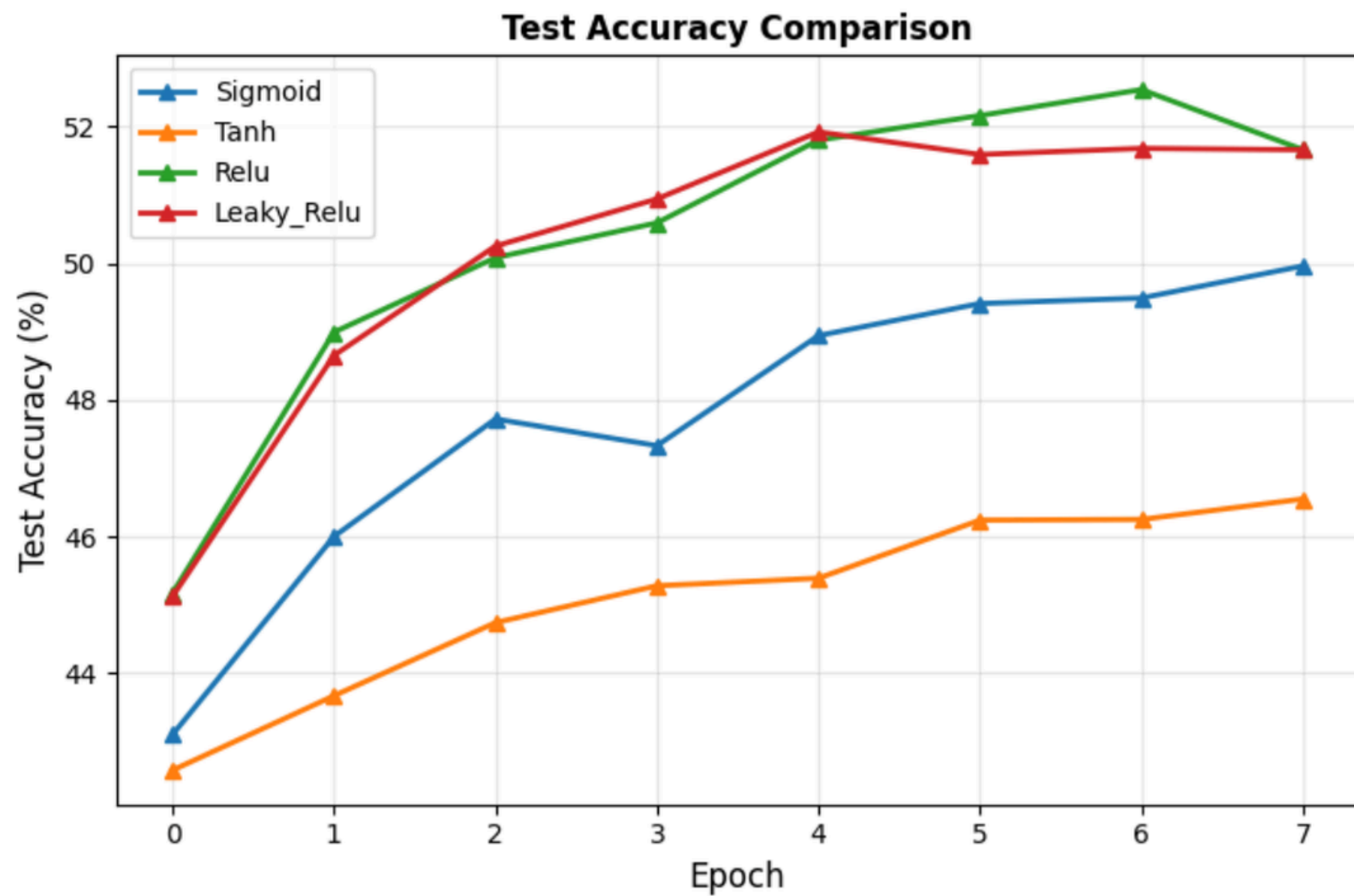
- If a **neuron outputs** only **negative** values, its **gradient** becomes **zero**.
- That **neuron** may **stop learning**.

But we can fix or reduce this using:

- **Leaky ReLU**

FAANG-level Answer

Activation Functions: Decision Maker



Our Mission Today: Create the Best Recipe for NN Soup

We have core ingredients to decide:

- The Architecture
- Activation Function
- Optimizer



Interview Moment: Optimizer Olympics



You are in a team that is working on movie genre classifier.

You want to do fast prototype experiment before jumping into actual big model training.

You are using a small **MLP** model.

BUT:

Training **loss** is **decreasing**...

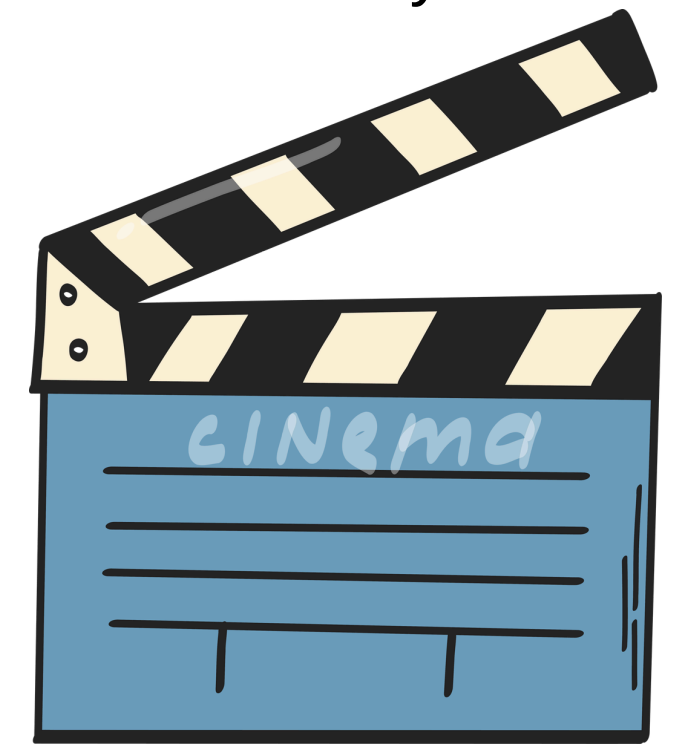
But **convergence** is **slow**

Currently, the model is trained using:

Vanilla SGD with $\text{lr}=0.001$, **no momentum**

During a cross-functional meeting which you are there as Senior ML engineer, they are expecting a clarification from you.

They want to know if they should **stick** to this setup or **try something else**.



Interview Moment: Optimizer Olympics



Adam is an **adaptive optimizer**, so it **automatically adjusts** the **learning rate**, which is better for faster convergence.

SGD is the **old basic optimizer** that doesn't adapt.

I would always **use Adam** because it's what everyone uses in practice and it just works better.

Medium Answer

Interview Moment: Optimizer Olympics



Adam is an **adaptive optimizer**, so it **automatically adjusts** the **learning rate**, which is better for faster convergence.

SGD is the **old basic optimizer** that doesn't adapt.

I would always **use Adam** because it's what everyone uses in practice and it just works better.

Medium Answer

Adam does two things at once:

It remembers which direction gradients have been pointing → momentum
it remembers how large those gradients have been → adaptive rates

A parameter that keeps receiving huge gradients gets its learning rate shrunk because it's already moving fast, no need to push harder.

A parameter that barely gets touched gets its learning rate boosted because it needs a bigger nudge to learn anything.

But I should make it clear that I am **not saying SGD is bad** and we shouldn't use it. There are definitely cases that we should **prefer SGD** based on the **literature** and my **practical experiences**.

FAANG-level Answer

Interview Moment: Optimizer Olympics



Adam is an **adaptive optimizer**, so it **automatically adjusts** the **learning rate**, which is better for faster convergence.

SGD is the **old basic optimizer** that doesn't adapt.

I would always **use Adam** because it's what everyone uses in practice and it just works better.

Medium Answer

We can use **Adam** when:

- **Rapid prototyping** (converges in fewer epochs)
- **High-dimensional**, sparse features(NLP embeddings)

But **SGD + Momentum** is also **great** choice if:

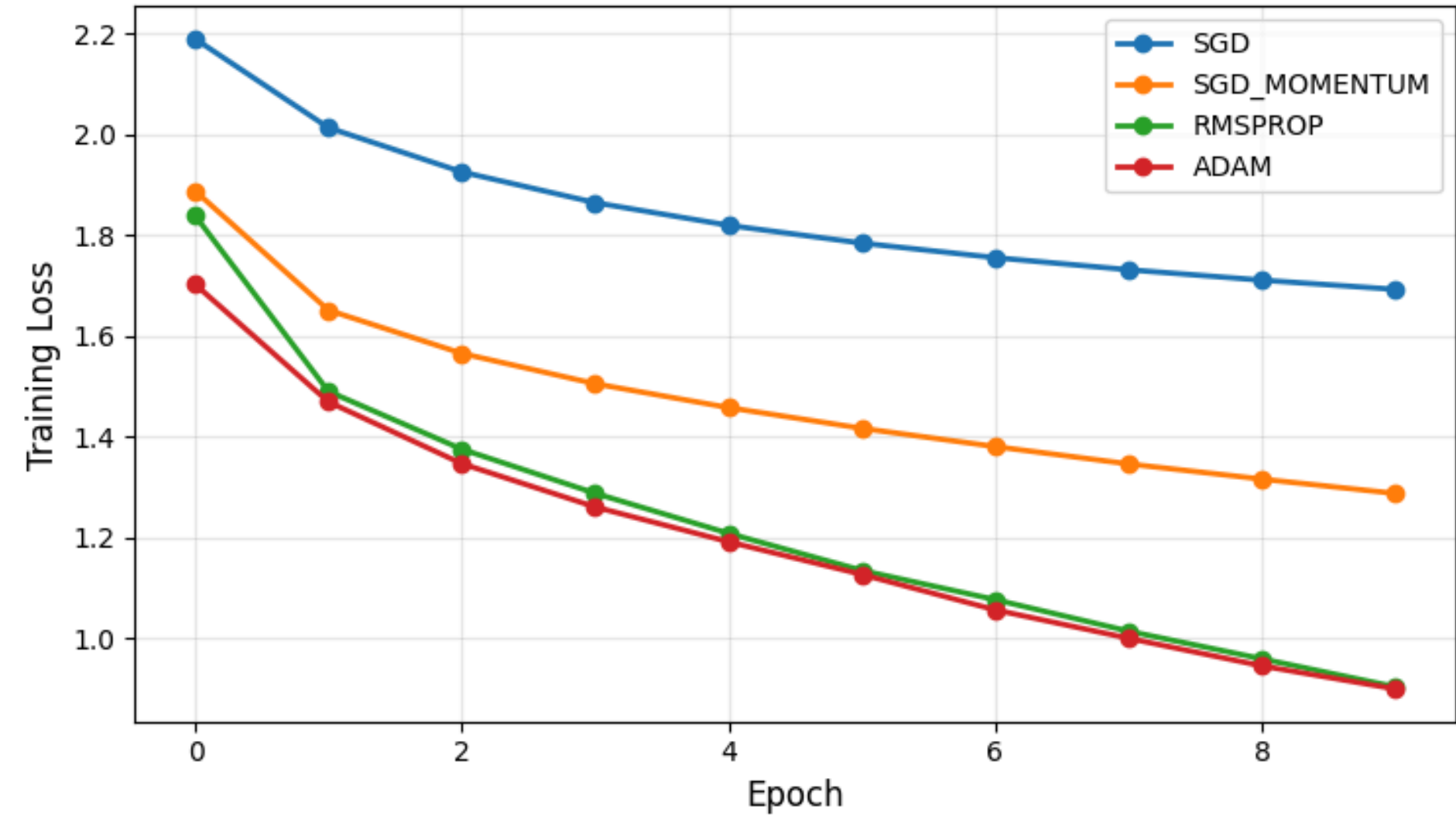
- You **need** the best possible **generalization** (research shows SGD often achieves 1-2% better test accuracy)
- Training large models where **memory matters** (Adam uses 2x memory)
- Computer **vision tasks** (**ResNets** often trained with SGD+momentum)

FAANG-level Answer

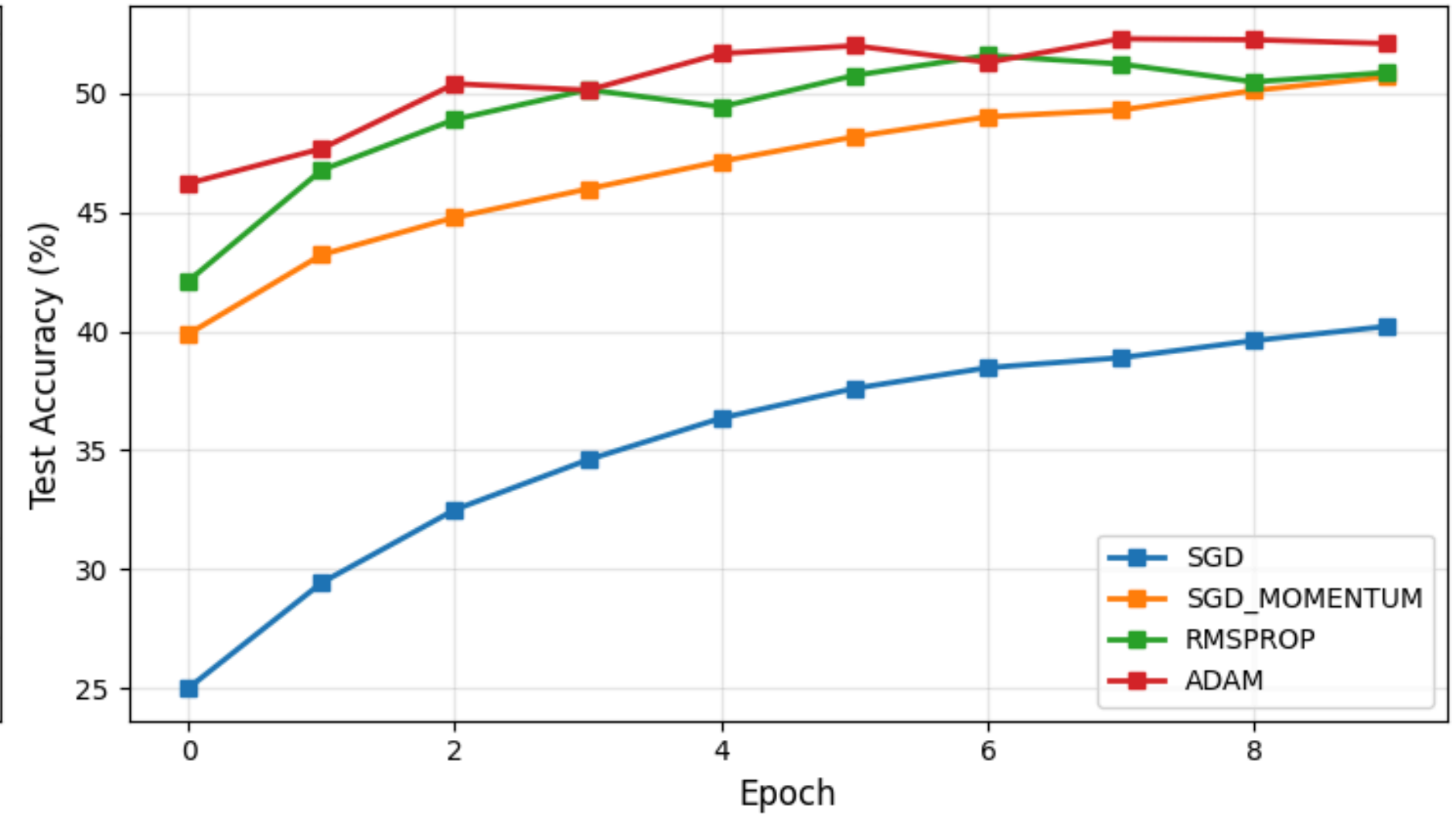
Optimizer Olympics

Optimizer Comparison (Same Model, Only Optimizer Changed)

Training Loss



Test Accuracy

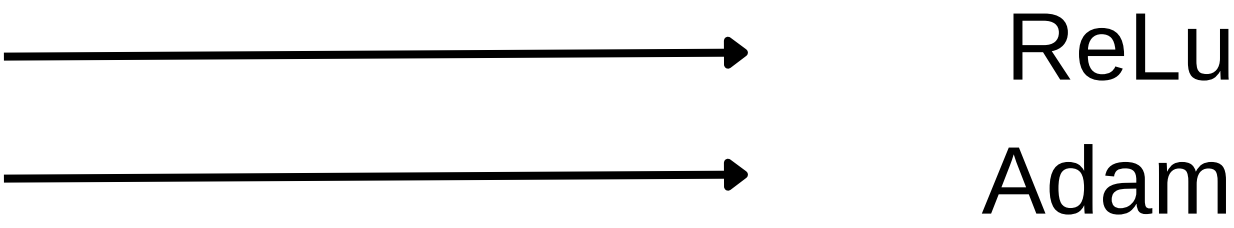


Our Mission Today: Create the Best Recipe for NN Soup

We have core ingredients to decide:

- The Architecture
- Activation Function
- Optimizer

- Learning Rate Schedules



Interview Moment: Learning Rate Schedulers



You're an ML engineer at Amazon working on the **recommendation system**.
Your team is about to launch a new **deep learning model** that predicts:
“**What** is the **probability** this user will start watching this title within 24 hours?”

You're **monitoring training** runs on the internal dashboard.
You notice something concerning:

Training **loss decreases** but incredibly **slow**,
The **test accuracy** plot seems **not smooth** at all there are **sharp jumps** and **downs**.

You receive message from your colleague on slack saying that:
“Just **increase** the **learning rate**. Maybe it's stuck in a **shallow region**.”
Another engineer texts: “We shouldn't be using a **fixed learning rate** at all. Let's add **step decay** or **cosine annealing**.”
The product launch clock is ticking.

Your tech lead messages you privately:
“Can you think through this carefully before we **rerun** another **expensive** 3-day **training job**?”

You first need to **explain** what is going on with this results and **suggest** an alternative **solution**.



Interview Moment: Learning Rate Schedulers



A **fixed learning rate** is **bad** because it doesn't **adapt** to the training process.

You should use a **learning rate scheduler** that decreases the learning rate over time, like **step decay**.

This helps the **model converge** better.

If the **model** is still **stuck**, maybe **try** a **different architecture** with more layers.

Good Answer

Interview Moment: Learning Rate Schedulers



A **fixed learning rate** is **bad** because it doesn't **adapt** to the training process.

You should use a **learning rate scheduler** that decreases the learning rate over time, like **step decay**.

This helps the **model converge** better.

If the **model** is still **stuck**, maybe **try** a **different architecture** with more layers.

Good Answer

The first thing comes to my mind is that having an issue with the learning rate. So I will just apply cosine annealing scheduler.

- Smooth transitions (no abrupt changes)
- Proven in practice (used in ResNet, EfficientNet papers)

I check if loss improves then I can say okay the problem was **LR** and hopefully it will be solved now.

If loss stays flat or I cannot get any improvement then I start considering other issues. I start checking gradient norms.

If $\text{grad_norm} < 1e-4$: Vanishing gradients (wrong activation)

If $\text{grad_norm} > 100$: Exploding gradients (LR too high or bad init)

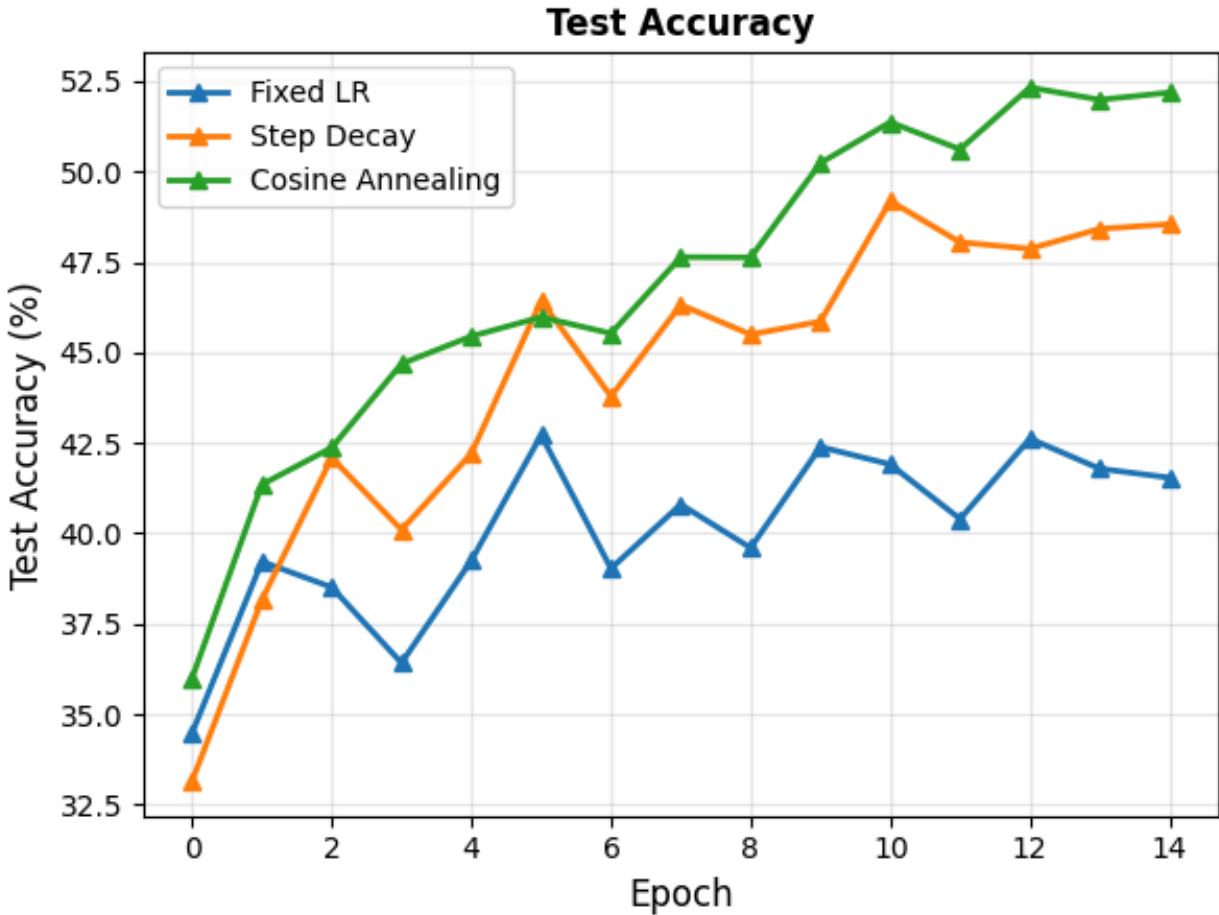
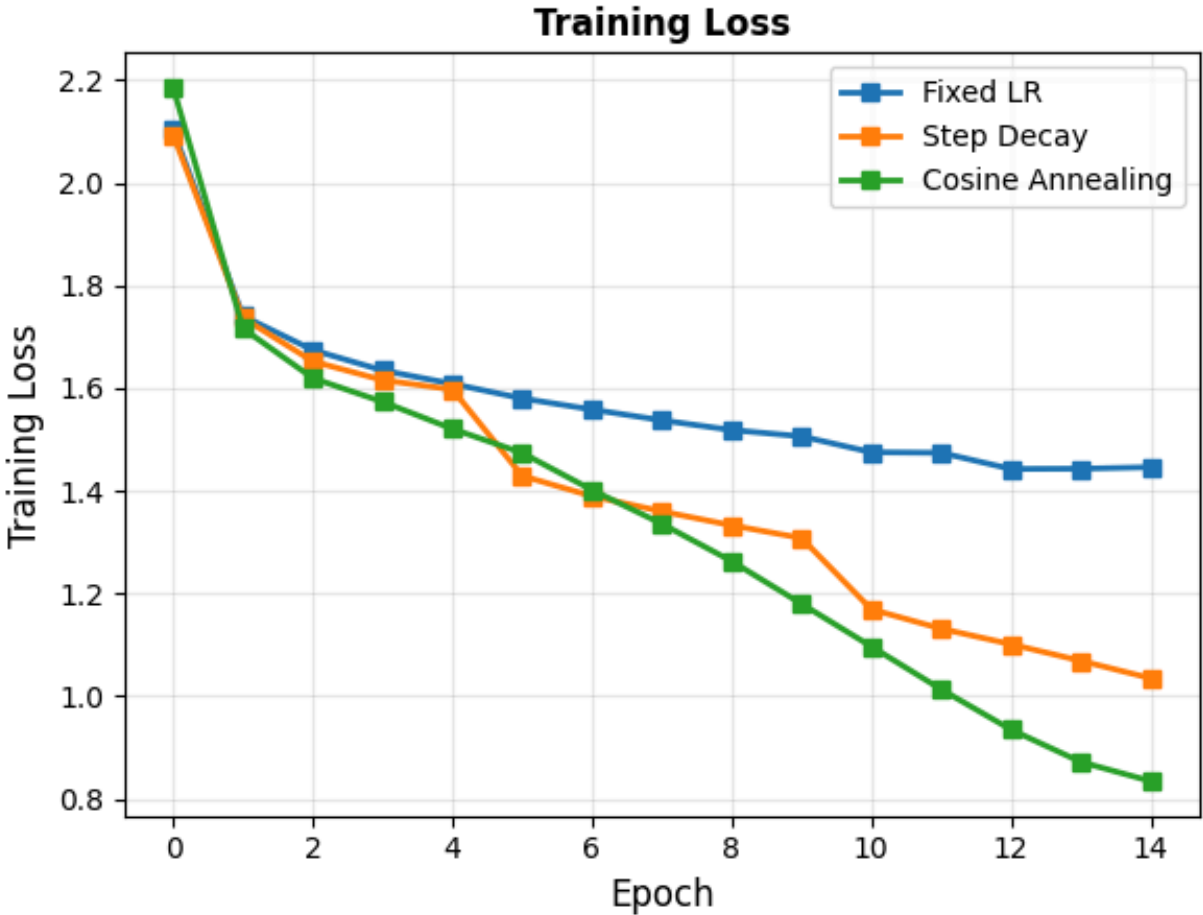
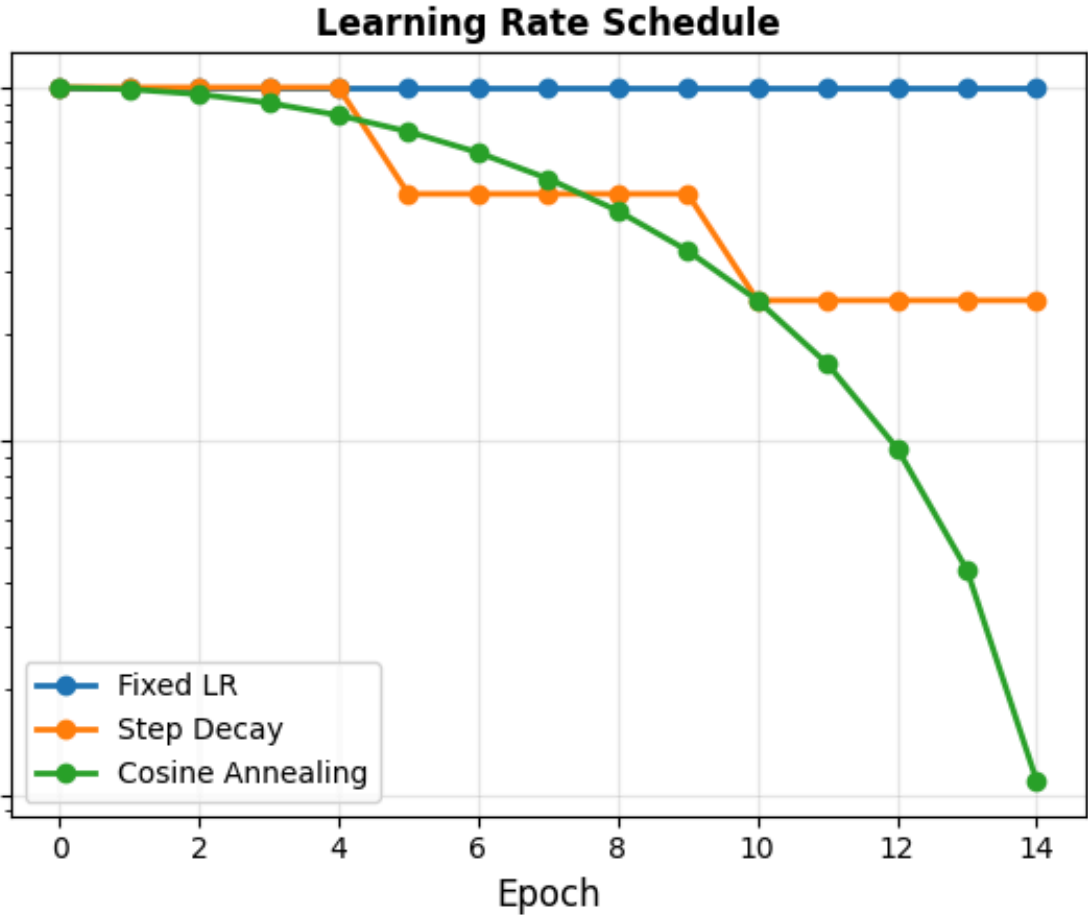
Then I can try different optimizer.

Even after that if I still have problems then I start considering the capacity of the model or the data.

FAANG-level Answer

Learning Rate Schedules

Learning Rate Schedule Comparison



Our Mission Today: Create the Best Recipe for NN Soup

We have core ingredients to decide:

- The Architecture
 - Activation Function  ReLu
 - Optimizer  Adam
 - Learning Rate Schedules  Cosine Decay
-
- Weight Initialization

Interview Moment: Start Smart with Weight Initialization



You've joined the On-Device Intelligence team at Apple.

The team is developing a **lightweight MLP** classifier for a new feature in iOS

A new engineer implements the MLP from scratch.

After 50 epochs:

Accuracy = 10%

Loss is not **decreasing at all**

(10% = random guessing for 10 classes.)



You check the initialization and find out that they used **zero initialization**.

Then you would like to **talk** with **engineer** and **explain** and **teach** them what **went wrong** with zero initialization.

So you visited engineer's office, what do you tell them?

Interview Moment: Start Smart with Weight Initialization



Zero initialization doesn't work because the **model can't learn**.

All **neurons become** the **same** and **gradients** don't **update** properly.

So we should use **random initialization** like **Xavier** or **He** because that's standard practice.

Good Answer

Interview Moment: Start Smart with Weight Initialization



Zero initialization doesn't work because the **model can't learn**.

All **neurons become** the **same** and **gradients** don't **update** properly.

So we should use **random initialization** like **Xavier** or **He** because that's standard practice.

Good Answer

Zero initialization **fails** due to **symmetry**.

If all weights in a layer are initialized to zero, every **neuron** in that layer **produces identical outputs**.

During backpropagation, they **receive identical gradients** and therefore **remain identical** after every update.

The **network** effectively **collapses** to **having one neuron** per layer.

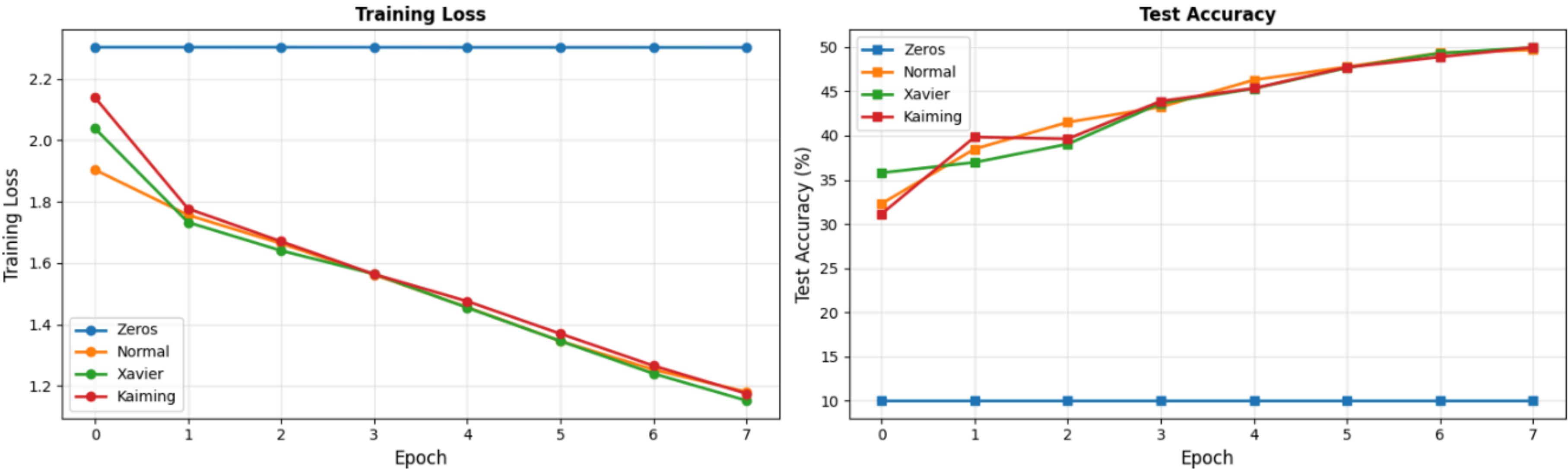
It **loses representational power** and cannot learn diverse features.

The solution is to **use** an **initialization strategy** that **preserves** the **scale** of **activations** and **gradients** across layers. For networks **with ReLU activations**, I would use He (**Kaiming**) **initialization**.

FAANG-level Answer

Start Smart with Weight Initialization

Weight Initialization Comparison



Our Mission Today: Create the Best Recipe for NN Soup

We have core ingredients to decide:

- The Architecture
 - Activation Function → ReLu
 - Optimizer → Adam
 - Learning Rate Schedules → Cosine Decay
 - Weight Initialization → Kaiming
-
- Sectar Ingredients???

Interview Moment: The Chef's Final Touches



You've joined the ML team at Amazon Robotics, which is developing a vision system for warehouse robots to identify packages on conveyor belts.

During training on the object classification model, you notice:

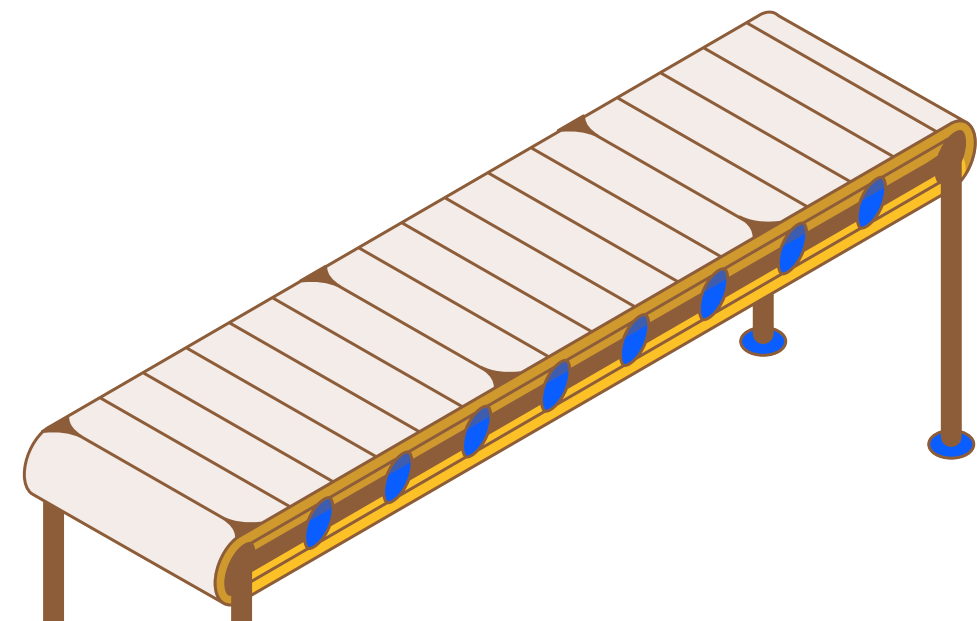
1. **Training accuracy** reaches **99%**, but **validation accuracy** is only **85%** → clear **overfitting**.
2. Validation accuracy **fluctuates** across batches, making **convergence slow**.

You are sure that you **applied everything** you **learned** from **MLE course**.

You **selected** the **right activation**, **right loss** for your problem, you know that your model **capacity** is **okay**, the **weight init** is **correct**, you use **learning rate scheduler**. **BUT** still encounter this **problem**.

In 3 hours you will have a meeting with the team and will show them these results, but you know that they will also ask you about the **possible solutions** that **you** are going to **implement** to **overcome** these issues.

What would you tell them ?



Interview Moment: The Chef's Final Touches



Dropout turns off some neurons randomly to stop overfitting.

Weight decay penalizes large weights directly in the optimizer.

Using these together should improve the model since they attack overfitting from different angles.

Good Answer

Interview Moment: The Chef's Final Touches



Dropout turns off some neurons randomly to stop overfitting.

Weight decay penalizes large weights directly in the optimizer.

Using these together should improve the model since they attack overfitting from different angles.

Good Answer

To reduce **overfitting**, one approach I would consider is combining two complementary regularization techniques:

Dropout randomly disables a fraction of neurons during training, forcing the network to learn distributed representations rather than memorizing specific patterns. This improves generalization to unseen packages.

Weight Decay (L2 regularization) penalizes large weight magnitudes directly in the optimizer at every update step.

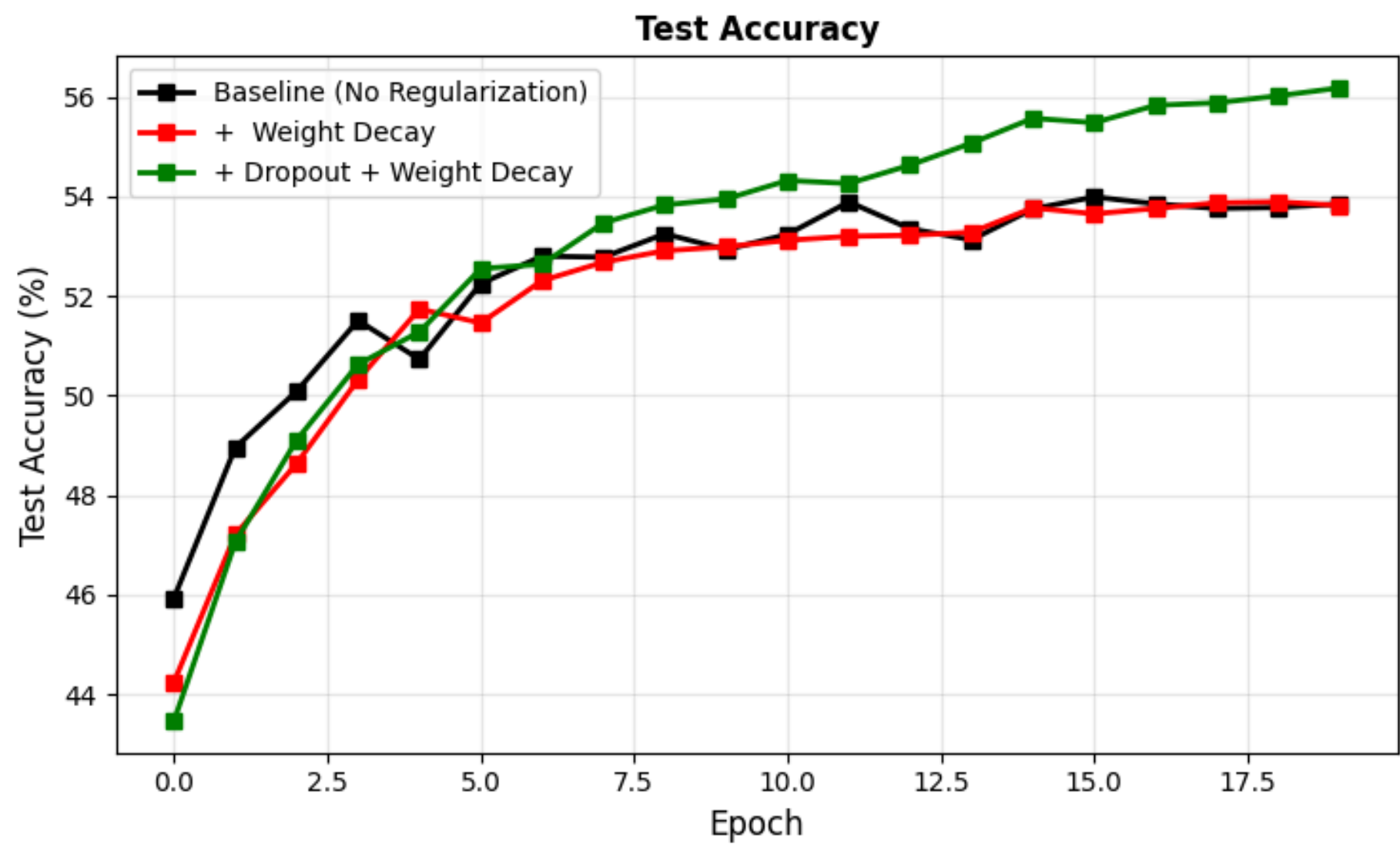
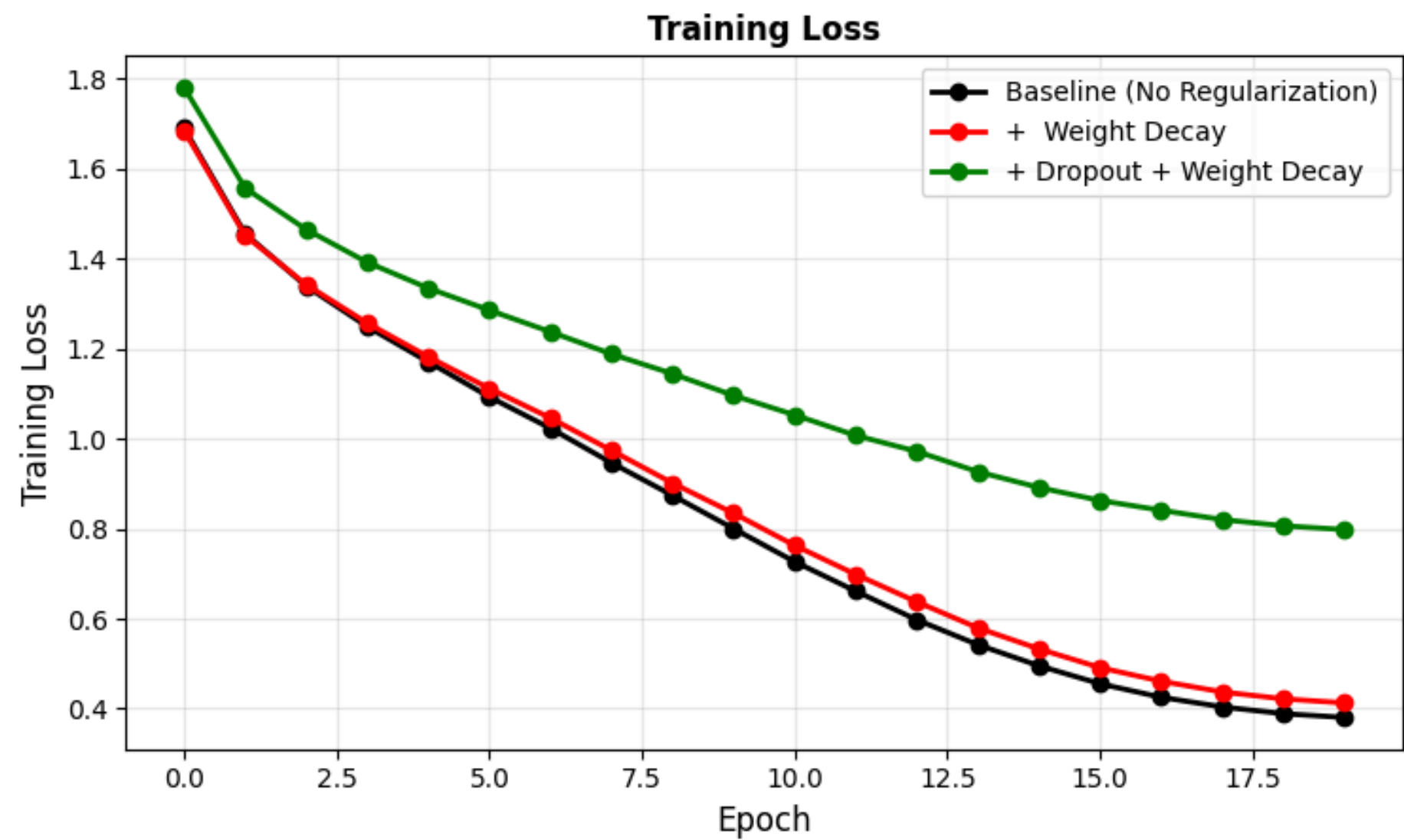
To improve the performance further, I could also try out several things like:

- Data augmentation,
- Early stopping,
- Batch norm,
- Increase the model capacity
- I could even try a complete different architecture that is more suitable for my problem 😊 (Like CNNs or transformers).

FAANG-level Answer

The Chef's Final Touches

Incremental Regularization: Building the Ultimate Model



Now is your turn

You can work in Teams

Total given time: 60 minutes

You need to do analysis as well and report the best model parameters